

CSE/ISyE 6740
Computational Data Analysis

Kernel Methods

09/23/2026

Kai Wang, Assistant Professor in Computational Science and Engineering

kwang692@gatech.edu

Outline

- **Supervised Learning**
 - Kernel methods
 - Nonlinear representation
 - Kernel function
 - Kernel regression
 - Kernel PCA

Nonlinear Representation

Linear and Ridge Regression (recap)

- Assume y is a linear function of $x \in \mathbb{R}^d$ (features) plus noise ϵ
$$y = \theta^\top x + \epsilon$$

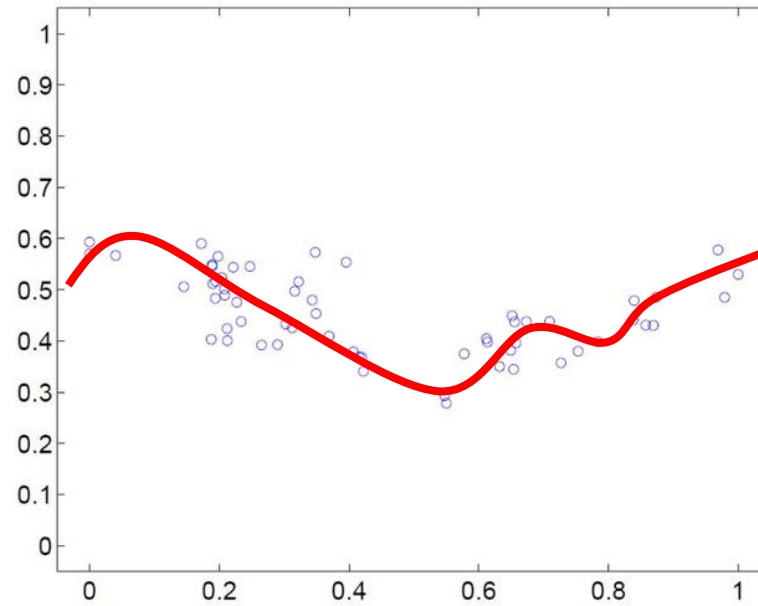
where ϵ is an error modeled as Gaussian $N(0, \sigma^2)$

- Given n data points, find θ that minimizes the mean square error

$$\hat{\theta} = \operatorname{argmin}_{\theta} L(\theta) + \frac{\lambda}{2} \|\theta\|^2 := \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \theta^\top x^{(i)})^2 + \frac{\lambda}{2} \|\theta\|^2$$

$$\begin{aligned} \frac{\partial L(\theta)}{\partial \theta} &= -\frac{2}{n} X^\top y + \frac{2}{n} X^\top X \theta + \lambda \theta = 0 \\ \Rightarrow \theta^* &= (X^\top X + \lambda I)^{-1} X^\top y \end{aligned}$$

Polynomial Regression



- What if we want to fit a d -dimensional polynomial regression model?

$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \cdots + \theta_d x^d + \epsilon$$

- Letting $\tilde{x} = (1, x, x^2, \dots, x^d)^\top$ and $\theta = (\theta_0, \theta_1, \theta_2, \dots, \theta_d)^\top$

- Do linear regression on $\tilde{x}$ to fit $y = \theta^\top \tilde{x}$

Least Mean Square Method for Polynomial

- Given n data points, find θ that minimizes the mean square error.

$$\hat{\theta} = \operatorname{argmin}_{\theta} L(\theta) := \frac{1}{n} \sum_{i=1}^n (y^i - \theta^{\top} \tilde{x}^i)^2$$

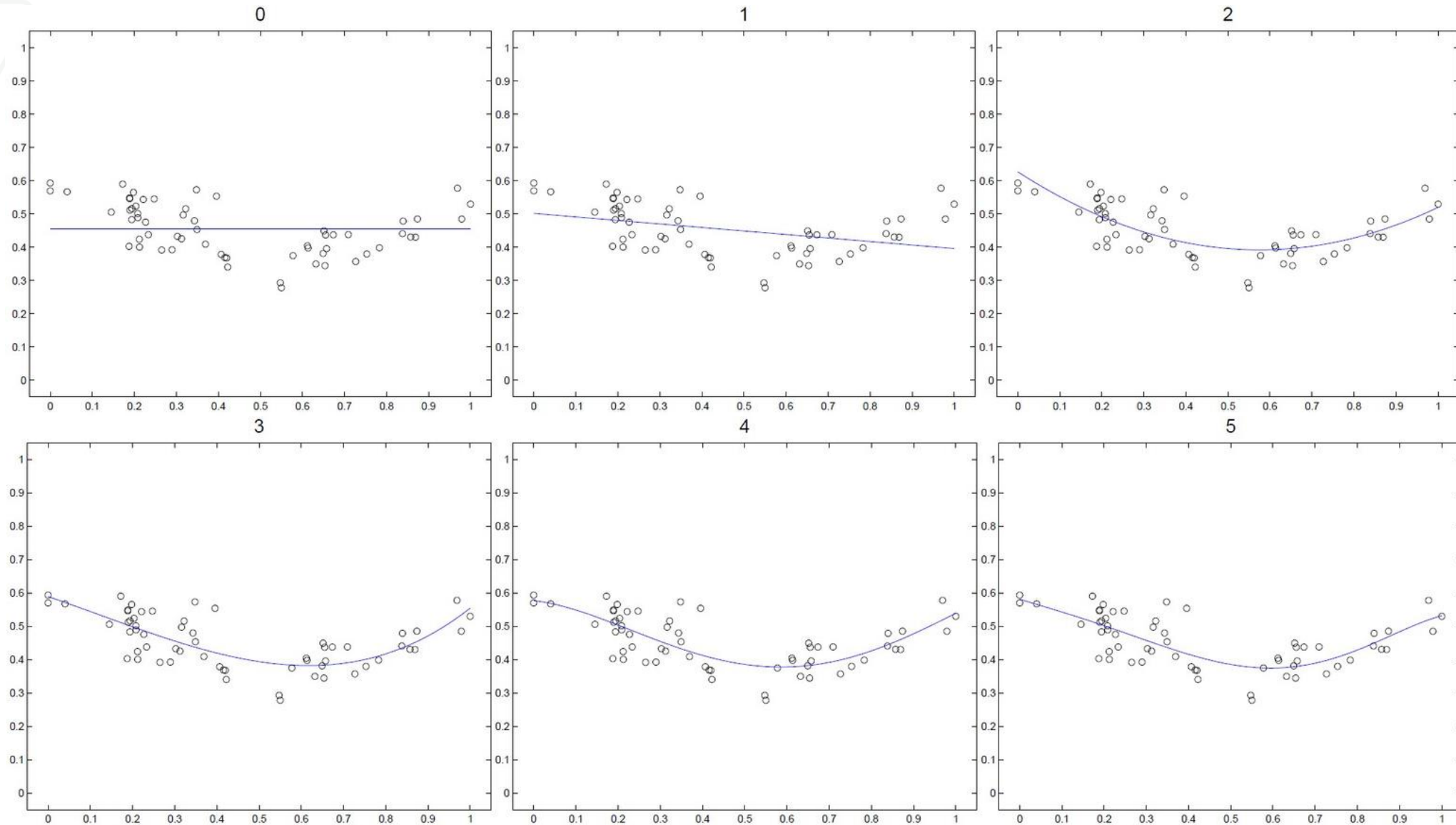
- Same as the linear regression (least mean square method), we can set gradient to 0 and write it in matrix form:

- Define $\tilde{X} = (\tilde{x}^{(1)}, \tilde{x}^{(2)}, \dots, \tilde{x}^{(n)})^{\top} \in \mathbb{R}^{n \times (d+1)}$, $y = (y^{(1)}, y^{(2)}, \dots, y^{(n)})^{\top} \in \mathbb{R}^n$, gradient becomes

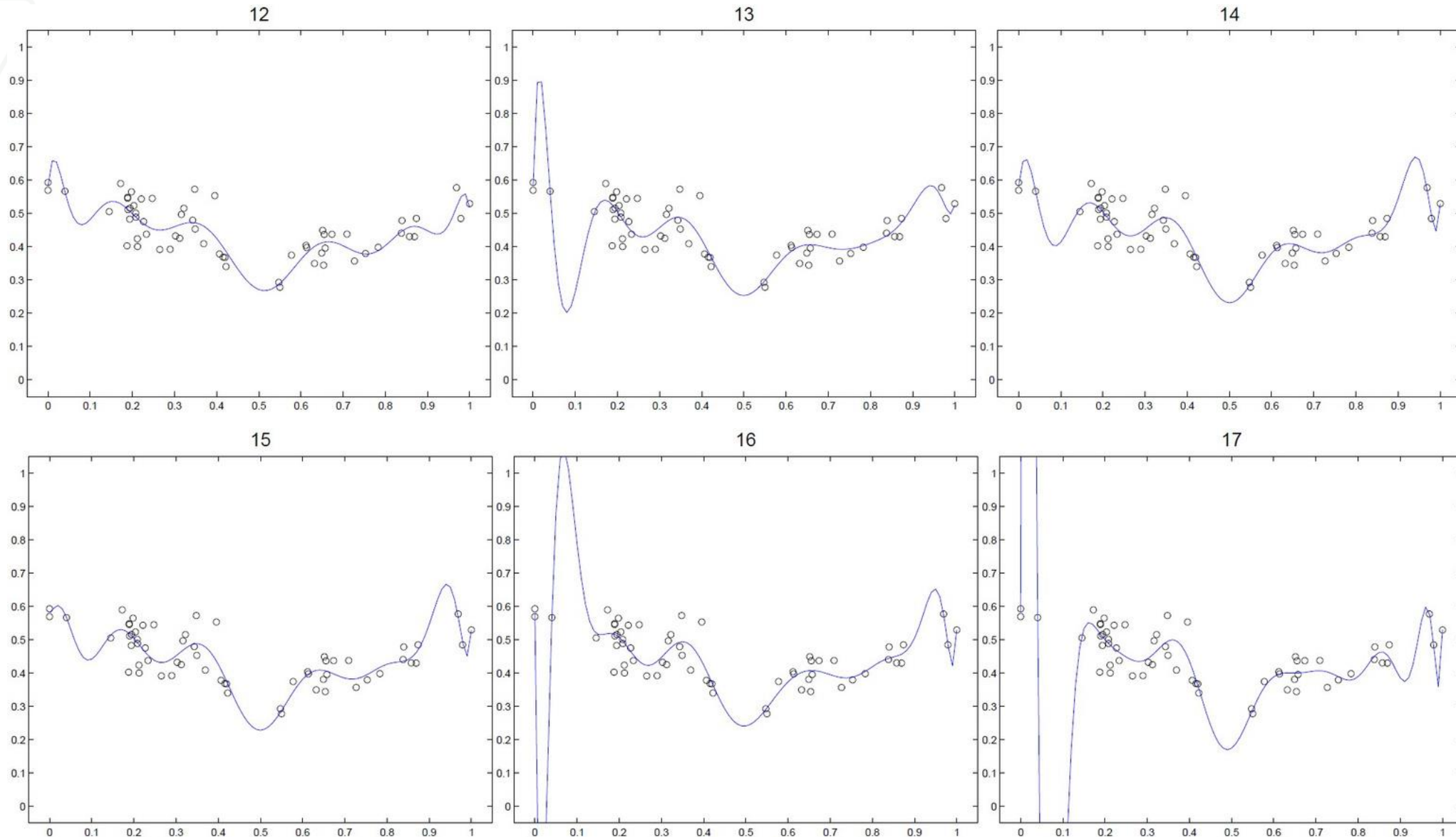
$$\frac{\partial L(\hat{\theta})}{\partial \theta} = -\frac{2}{n} \tilde{X}^{\top} y + \frac{2}{n} \tilde{X}^{\top} \tilde{X} \theta = 0 \quad \Rightarrow \quad \hat{\theta} = (\tilde{X}^{\top} \tilde{X})^{-1} \tilde{X}^{\top} y$$

- Picking different maximal degree d for the polynomial will give a different solution.

Increasing the Maximal Degree



Increasing the Maximal Degree



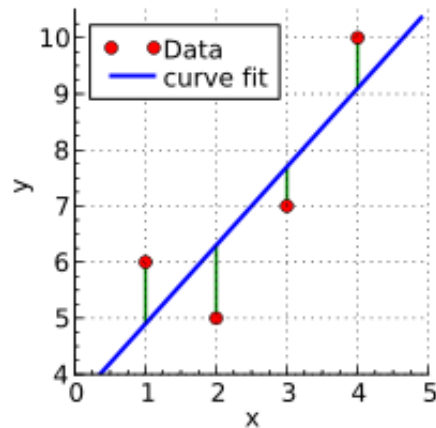
Linear Regression v.s. Polynomial Regression

- Given a training dataset $D_{train} = \{(x^{(i)}, y^{(i)})\}_{i \in \{1, 2, \dots, n\}}$

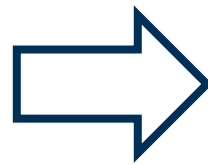
- Linear regression**

$$y = \theta^\top x + \epsilon$$

where $x \in \mathbb{R}, y \in \mathbb{R}$



$x \in \mathbb{R}$



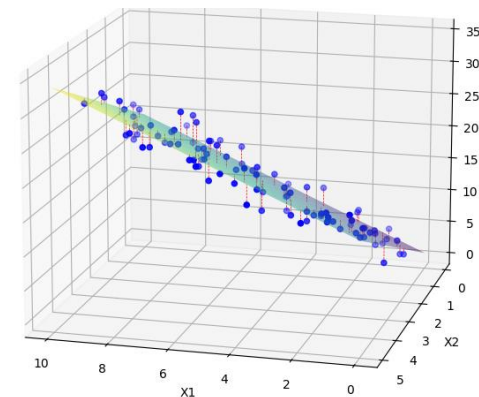
- d -dim polynomial regression**

$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_d x^d + \epsilon$$
$$= \theta^\top \tilde{x}$$

where $\tilde{x} = [1, x, x^2, \dots, x^d]^\top \in \mathbb{R}^{d+1}, y \in \mathbb{R}$

$$\tilde{x} = \phi(x) = [1, x, x^2, \dots, x^d]^\top \in \mathbb{R}^{d+1}$$

Now it becomes a high-dimensional “linear” regression problem on $\tilde{x}$



High-dimensional Polynomial Regression

- Given a training dataset $D_{train} = \{(x^{(i)}, y^{(i)})\}_{i \in \{1, 2, \dots, n\}}$

- Linear regression**

$$y = \theta^\top x + \epsilon$$

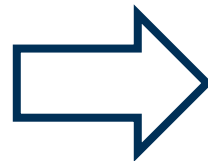
where $x = [x_1, x_2, \dots, x_d]^\top \in \mathbb{R}^d, y \in \mathbb{R}$

- k -dim polynomial regression**

$$\begin{aligned} y &= \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d \\ &\quad + \theta_{1,1} x_1 x_1 + \theta_{1,2} x_1 x_2 + \dots \\ &\quad + \theta_{i_1 i_2, \dots, i_m} x_{i_1} x_{i_2} \dots x_{i_d} + \dots + \epsilon \\ &= \theta^\top \tilde{x} \end{aligned}$$

where $\tilde{x} = [1, x_1, \dots, x_{i_1} x_{i_2} \dots x_{i_k}]^\top \in \mathbb{R}^{d'}, y \in \mathbb{R}$

$x \in \mathbb{R}^d$



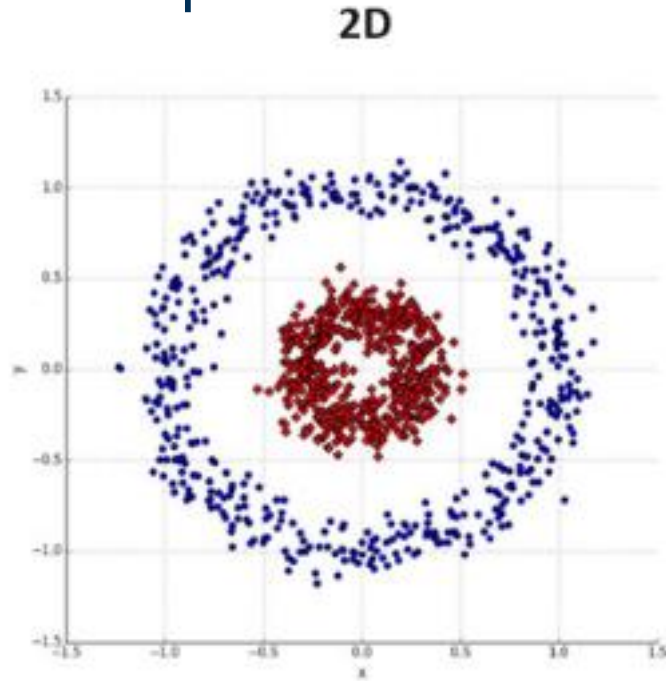
$\tilde{x} = \phi(x) = [1, x_1, \dots, x_{i_1} x_{i_2} \dots x_{i_k}]^\top \in \mathbb{R}^{d'}$

You will have all possible polynomials up to degree k , i.e., $\binom{k+d-1}{k}$ number of features

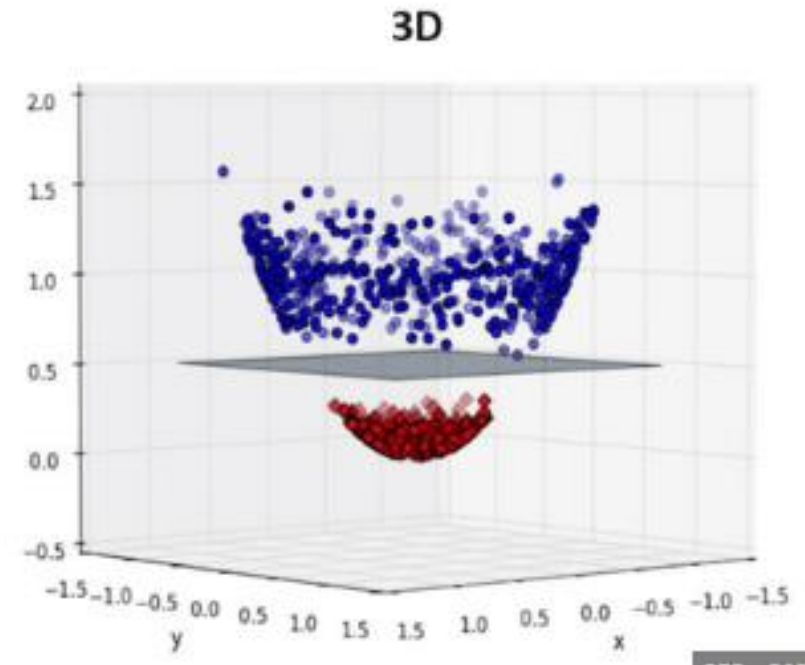
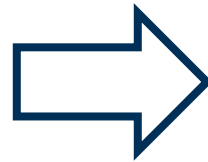
Now it becomes a high-dimensional “linear” regression problem on $\tilde{x}$

Can We Generalize this Idea?

- Define a non-linear mapping $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$ to map from d -dimensional points to d' -dimensional points.



$$x \in \mathbb{R}^d$$



$$\tilde{x} = \phi(x) \in \mathbb{R}^{d'}$$

Non-linear Regression (given embedding $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$)

- Given a training dataset $D_{train} = \{(x^{(i)}, y^{(i)})\}_{i \in \{1, 2, \dots, n\}}$
- Construct a new dataset $D'_{train} = \{(\phi(x^{(i)}), y^{(i)})\}$
- Do ridge regression on D'_{train}
 - Define $\tilde{X} = [\phi(x^{(1)}), \phi(x^{(2)}), \dots, \phi(x^{(n)})]^\top, y = [y^{(1)}, y^{(2)}, \dots, y^{(n)}]^\top$
 - We can solve $\theta^* = (\tilde{X}^\top \tilde{X} + \lambda I)^{-1} \tilde{X}^\top y$

Kernel Function

Can We Avoid Expanding the Features?

- Kernel functions!
- Kernel $k: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ measures the nonlinear similarity between two points $x^{(i)}, x^{(j)}$
 - The main idea is to use $k(x^{(i)}, x^{(j)}) = \langle \phi(x^{(i)}), \phi(x^{(j)}) \rangle = \phi(x^{(i)})^\top \phi(x^{(j)})$ to replace all inner products.
 - Hope that we can just use k to avoid ϕ

Example: Kernel SVM

- We can apply embedding $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$ to the dual problem of SVM.
- The inner products in the dual formulation can be replaced by kernel function.
- The advantage is that we don't need to know/specify the embedding ϕ .

Dual problem

$$\begin{aligned} \max_{\alpha} g(\alpha) &:= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y^i y^j (x^{i\top} x^j) \\ \text{s.t. } C &\geq \alpha_i \geq 0 \quad \forall i, \quad \sum_{i=1}^n \alpha_i y^i = 0 \end{aligned}$$

Dual problem + embedding ϕ

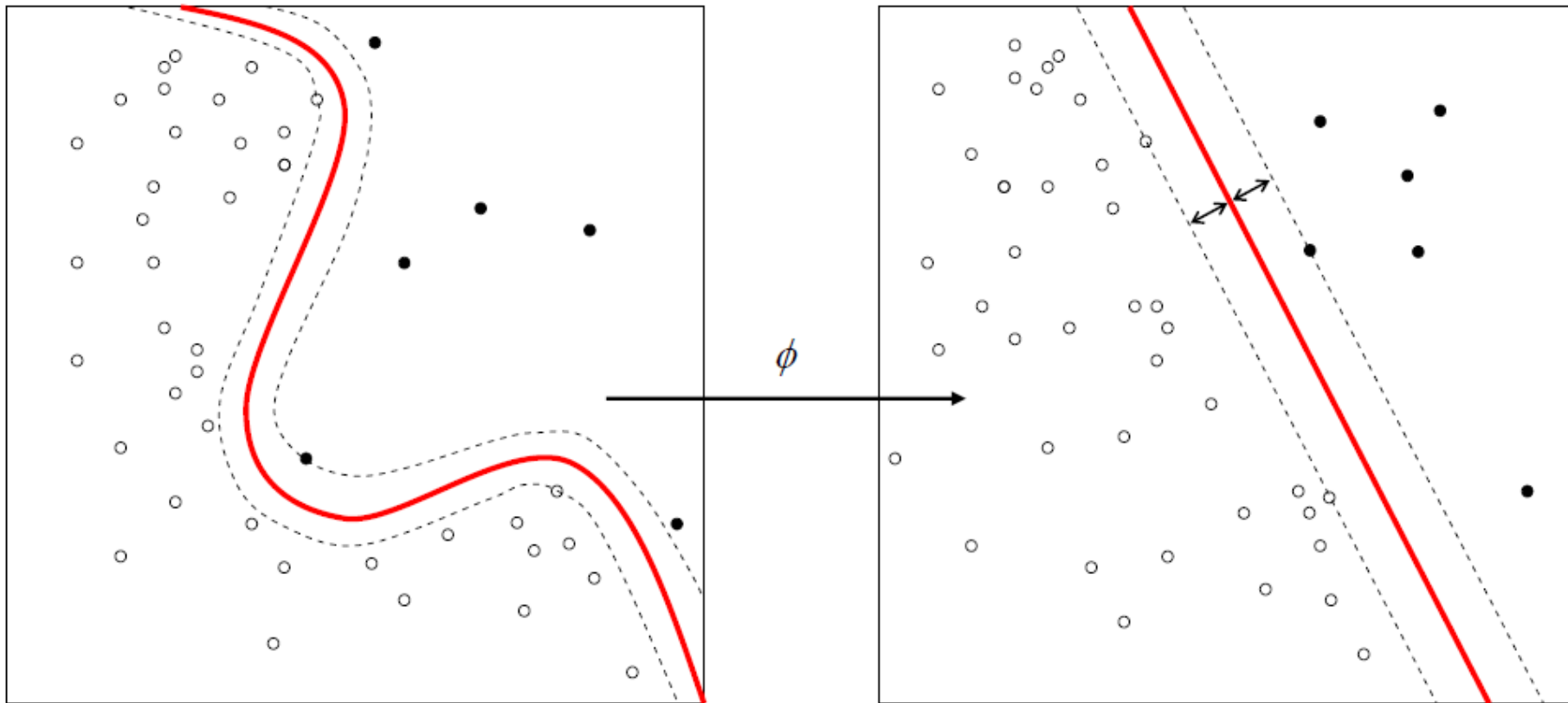
$$\begin{aligned} \max_{\alpha} g(\alpha) &:= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y^i y^j (\phi(x^{(i)})^\top \phi(x^{(j)})) \\ \text{s.t. } C &\geq \alpha_i \geq 0 \quad \forall i, \quad \sum_{i=1}^n \alpha_i y^i = 0 \end{aligned}$$

Dual problem + kernel k

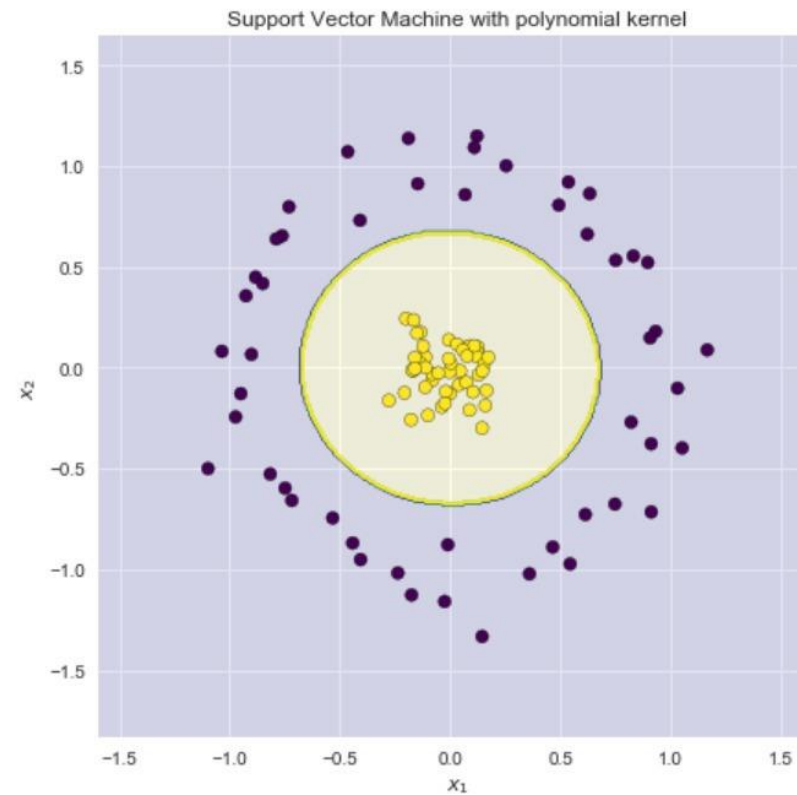
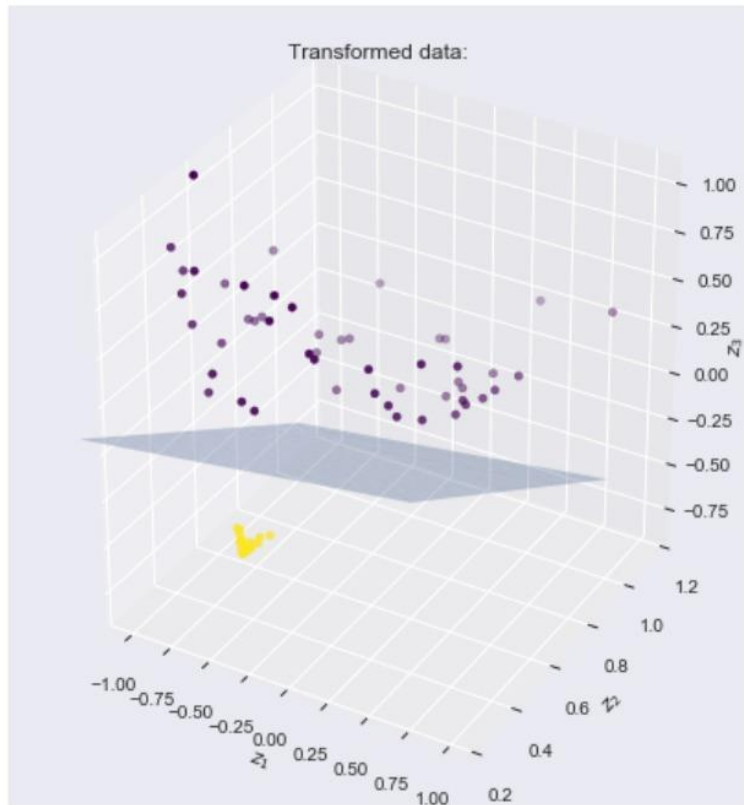
$$\begin{aligned} \max_{\alpha} g(\alpha) &:= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y^i y^j k(x^{(i)}, x^{(j)}) \\ \text{s.t. } C &\geq \alpha_i \geq 0 \quad \forall i, \quad \sum_{i=1}^n \alpha_i y^i = 0 \end{aligned}$$

Kernel SVM Example

- Kernel SVM implicitly maps data into a nonlinear feature space
- Find linear decision boundary in the new space



Kernel SVM Example



Common Kernel Functions

- Polynomial of degree d
 - $k(x, x') = (x^\top x')^d$
- Polynomial of degree up to d
 - $k(x, x') = (x^\top x' + c)^d$
- Exponential kernel (infinite degree polynomials)
 - $k(x, x') = \exp(s \cdot x^\top x') \approx 1 + sx^\top x' + \frac{s^2(x^\top x')^2}{2} + \frac{s^3(x^\top x')^3}{3!} + \dots$
- Gaussian RBF kernel
 - $k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$
- Laplace kernel
 - $k(x, x') = \exp\left(-\frac{\|x - x'\|}{2\sigma^2}\right)$
- Exponentiated distance
 - $k(x, x') = \exp\left(-\frac{d(x, x')^2}{s^2}\right)$

What $k(x, x')$ can be Called a Kernel Function?

- $k(x, x') = \phi(x)^\top \phi(x')$ (but we don't know ϕ)

- Given a dataset $D = \{(x^{(i)}, y^{(i)})\}_{i \in \{1, 2, \dots, n\}}$

y is optional
here

- Compute pairwise kernel function $k(x^{(i)}, x^{(j)})$ and form a $n \times n$ kernel matrix (Gram matrix)

$$K = \begin{pmatrix} k(x^{(1)}, x^{(1)}) & k(x^{(1)}, x^{(2)}) & \dots & k(x^{(1)}, x^{(n)}) \\ k(x^{(2)}, x^{(1)}) & k(x^{(2)}, x^{(2)}) & & k(x^{(2)}, x^{(n)}) \\ \vdots & & \ddots & \\ k(x^{(n)}, x^{(1)}) & k(x^{(n)}, x^{(2)}) & & k(x^{(n)}, x^{(n)}) \end{pmatrix}$$

- $k(x, x')$ is a kernel function iff matrix K is positive semidefinite
 - $\forall v \in \mathbb{R}^n, v^\top K v \geq 0$

Kernel Regression

Kernel Regression

- Given a training dataset $D_{train} = \{(x^{(i)}, y^{(i)})\}_{i \in \{1, 2, \dots, n\}}$
- Construct a new dataset $D'_{train} = \{(\phi(x^{(i)}), y^{(i)})\}$
- Do ridge regression on D'_{train}
 - Define $\tilde{X} = [\phi(x^{(1)}), \phi(x^{(2)}), \dots, \phi(x^{(n)})]^\top, y = [y^{(1)}, y^{(2)}, \dots, y^{(n)}]^\top$
 - We can solve $\theta^* = (\tilde{X}^\top \tilde{X} + \lambda I)^{-1} \tilde{X}^\top y$
- Let's see what we can do to express this in terms of K

Kernel Regression

- **Matrix inversion lemma** ($\forall B \in \mathbb{R}^{n \times m}$), we have:
$$(BB^\top + \lambda I)^{-1}B = B(B^\top B + \lambda I)^{-1}$$
- Evaluate ridge regression solution $\theta^* = (\tilde{X}^\top \tilde{X} + \lambda I)^{-1} \tilde{X}^\top y$ on a new point x
 - **Prediction:** $\theta^{*\top} \phi(x) = \phi(x)^\top \theta^* = \phi(x)^\top (\tilde{X}^\top \tilde{X} + \lambda I)^{-1} \tilde{X}^\top y = \phi(x)^\top \tilde{X}^\top (\tilde{X} \tilde{X}^\top + \lambda I)^{-1} y$
$$= k_x^\top (K + \lambda I)^{-1} y$$
- Where $K = [k(x^{(i)}, x^{(j)})]_{i,j \in [n]} \in \mathbb{R}^{n \times n}$, and $k_x = (k(x^{(i)}, x))_{i \in [n]} \in \mathbb{R}^{n \times 1}$

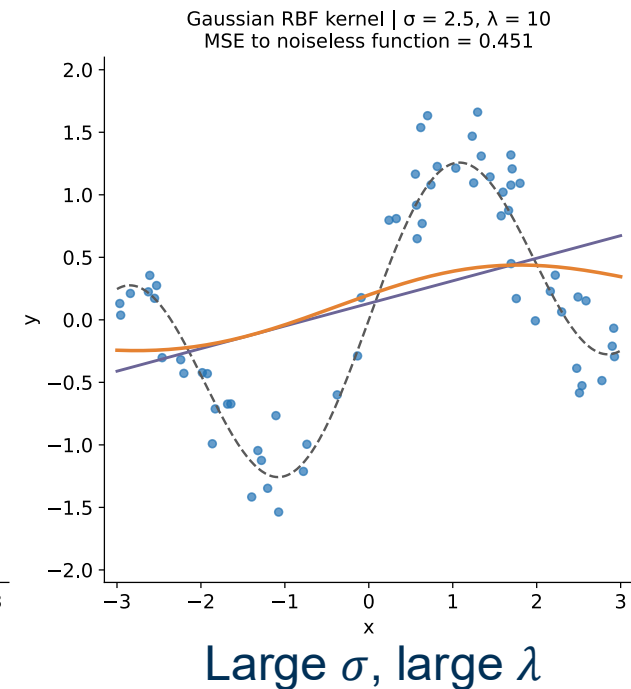
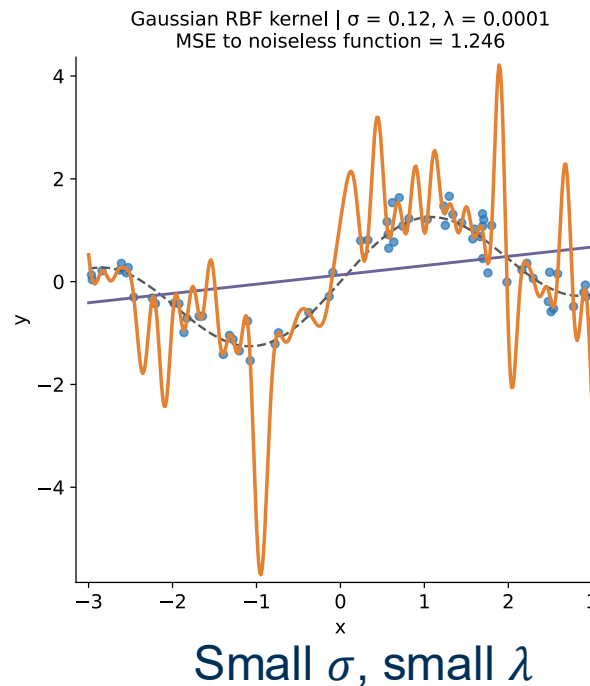
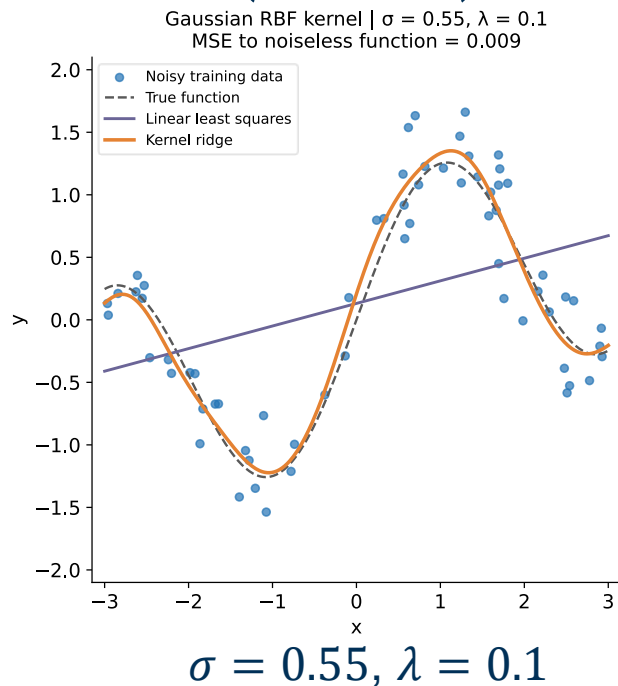
Kernel Regression

1. Given a training dataset $D_{train} = \{(x^{(i)}, y^{(i)})\}_{i \in \{1, 2, \dots, n\}}$
2. Compute the Kernel matrix: $K = [k(x^{(i)}, x^{(j)})]_{i, j \in [n]} \in \mathbb{R}^{n \times n}$
3. For any new point x , compute $k_x = \left(k(x^{(i)}, x)\right)_{i \in [n]} \in \mathbb{R}^{n \times 1}$
4. Prediction: $k_x^\top (K + \lambda I)^{-1} y$

Kernel Regression Example

- Use Gaussian RBF kernel

- $k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$



- Use cross-validation to choose parameters

Questions

- Is the regression method using a given embedding model ϕ a parametric model or non-parametric model?
- Is the kernel regression a parametric model or non-parametric model?
- What is the computation cost of kernel regression?
 - Prediction of x is $k_x^\top (K + \lambda I)^{-1} y$

Kernel PCA

Principal Component Analysis (recap)

- Given n data points, $\{x^{(1)}, x^{(2)}, \dots, x^{(n)}\} \in \mathbb{R}^d$
- **Step 1:** estimate the mean and covariance matrix from the data:
 - $\mu = \frac{1}{n} \sum_{i=1}^n x^{(i)}$ and $C = \frac{1}{n} \sum_{i=1}^n (x^{(i)} - \mu)(x^{(i)} - \mu)^\top$
- **Step 2:** take the eigenvectors $w^1, w^2, \dots$ of C corresponding to the largest eigenvalue λ_1 , the second largest eigenvalue $\lambda_2, \dots$
- **Step 3:** compute reduced representation

Can we apply the
kernel methods to
PCA?

$$z^i = \begin{pmatrix} \frac{w^1{}^\top (x^{(i)} - \mu)}{\sqrt{\lambda_1}} \\ \frac{w^2{}^\top (x^{(i)} - \mu)}{\sqrt{\lambda_2}} \\ \vdots \end{pmatrix}$$

Kernel PCA

- Given n data points, $\{x^{(1)}, x^{(2)}, \dots, x^{(n)}\} \in \mathbb{R}^d$
- **Step 1:** estimate the mean and covariance matrix from the data:
 - $\mu = \frac{1}{n} \sum_{i=1}^n \phi(x^{(i)})$ and $C = \frac{1}{n} \sum_{i=1}^n (\phi(x^{(i)}) - \mu)(\phi(x^{(i)}) - \mu)^\top$
- **Step 2:** take the eigenvectors $w^1, w^2, \dots$ of C corresponding to the largest eigenvalue λ_1 , the second largest eigenvalue $\lambda_2, \dots$
- **Step 3:** compute reduced representation

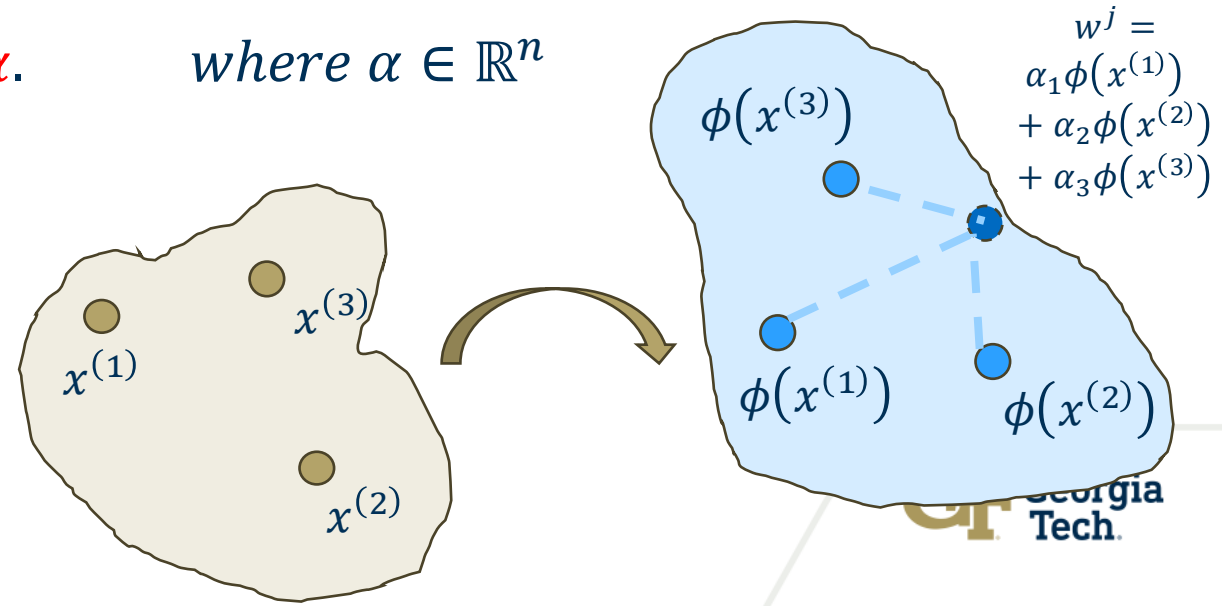
$$z^i = \begin{pmatrix} \frac{w^1{}^\top (\phi(x^{(i)}) - \mu)}{\sqrt{\lambda_1}} \\ \frac{w^2{}^\top (\phi(x^{(i)}) - \mu)}{\sqrt{\lambda_2}} \\ \vdots \end{pmatrix}$$

Kernel PCA

- We want to do eigen-decomposition on $\tilde{C} = \frac{1}{n} \sum_{i=1}^n (\phi(x^{(i)}) - \mu)(\phi(x^{(i)}) - \mu)^\top$
- For simplicity, let's assume $x^{(i)}$ and $\phi(x^{(i)})$ are normalized to have 0 mean.
 - $\tilde{C} = \frac{1}{n} \sum_{i=1}^n \phi(x^{(i)})\phi(x^{(i)})^\top = \frac{1}{n} \tilde{X}^\top \tilde{X}$
- **Trick:** express the principal component w^j as a linear combination of the data points

$$w^j = \sum_{i=1}^n \alpha_i \phi(x^{(i)}) = \tilde{X}^\top \alpha.$$

where $\alpha \in \mathbb{R}^n$



- Can we use this to find $\tilde{C}w^j = \lambda^j w^j$?

Kernel PCA

- We have:

$$\lambda_j w^j = C w^j = \frac{1}{n} \tilde{X}^\top \tilde{X} w^j = \frac{1}{n} \tilde{X}^\top \tilde{X} \tilde{X}^\top \alpha = \frac{1}{n} \tilde{X}^\top K \alpha$$

- Multiple both side by $\tilde{X}$ on the left, we get

$$\frac{1}{n} \tilde{X} \tilde{X}^\top K \alpha = \lambda_j \tilde{X} w^j = \lambda_j \tilde{X} \tilde{X}^\top \alpha = \lambda_j K \alpha$$

- Therefore, we have:

$$\frac{1}{n} K^2 \alpha = \lambda_j K \alpha$$

Kernel PCA

- We can simply write

$$K\alpha^j = n\lambda_j\alpha^j$$

Are they orthonormal?

- I.e. α^j is an eigenvector of the kernel matrix K , with eigenvalues $n\lambda_j$
- Often, we want to keep the principal component $w^j = \tilde{X}^\top \alpha^j$ to have norm 1 (normal)

$$\|w^j\|^2 = \alpha^{j\top} \tilde{X} \tilde{X}^\top \alpha^j = \alpha^{j\top} K \alpha^j = 1 \text{ (scale } \alpha^j \text{ to make it norm 1)}$$

- For any data point x , we can represent it as

$$z = \begin{pmatrix} \phi(x)^\top w^1 \\ \phi(x)^\top w^2 \\ \dots \\ \phi(x)^\top w^n \end{pmatrix} = \begin{pmatrix} \phi(x)^\top \tilde{X}^\top \alpha^1 \\ \phi(x)^\top \tilde{X}^\top \alpha^2 \\ \dots \\ \phi(x)^\top \tilde{X}^\top \alpha^n \end{pmatrix} = \begin{pmatrix} k_x^\top \alpha^1 \\ k_x^\top \alpha^2 \\ \dots \\ k_x^\top \alpha^n \end{pmatrix}$$

Kernel PCA

- When $\phi(x^{(i)})$ does not have 0 mean, set $\tilde{\phi}(x^{(i)}) = \phi(x^{(i)}) - \frac{1}{n} \sum_{k=1}^n \phi(x^{(k)})$ and redo the same process

- The kernel of $\tilde{\phi}$ becomes

$$\begin{aligned}\tilde{K}(x^{(i)}, x^{(j)}) &= \tilde{\phi}(x^{(i)})^\top \tilde{\phi}(x^{(j)}) \\ &= \left(\phi(x^{(i)}) - \frac{1}{n} \sum_{k=1}^n \phi(x^{(k)}) \right)^\top \left(\phi(x^{(j)}) - \frac{1}{n} \sum_{k'=1}^n \phi(x^{(k')}) \right) \\ &= K(x^{(i)}, x^{(j)}) - \frac{1}{n} \sum_{k'=1}^n K(x^{(i)}, x^{(k')}) - \frac{1}{n} \sum_{k=1}^n K(x^{(k)}, x^{(j)}) + \frac{1}{n^2} \sum_{k,k'} K(x^{(k)}, x^{(k')})\end{aligned}$$

$$\tilde{K} = K - \frac{1}{n} K \mathbf{1}_{n \times n} - \frac{1}{n} \mathbf{1}_{n \times n} K + \frac{1}{n^2} \mathbf{1}_{n \times n} K \mathbf{1}_{n \times n}$$

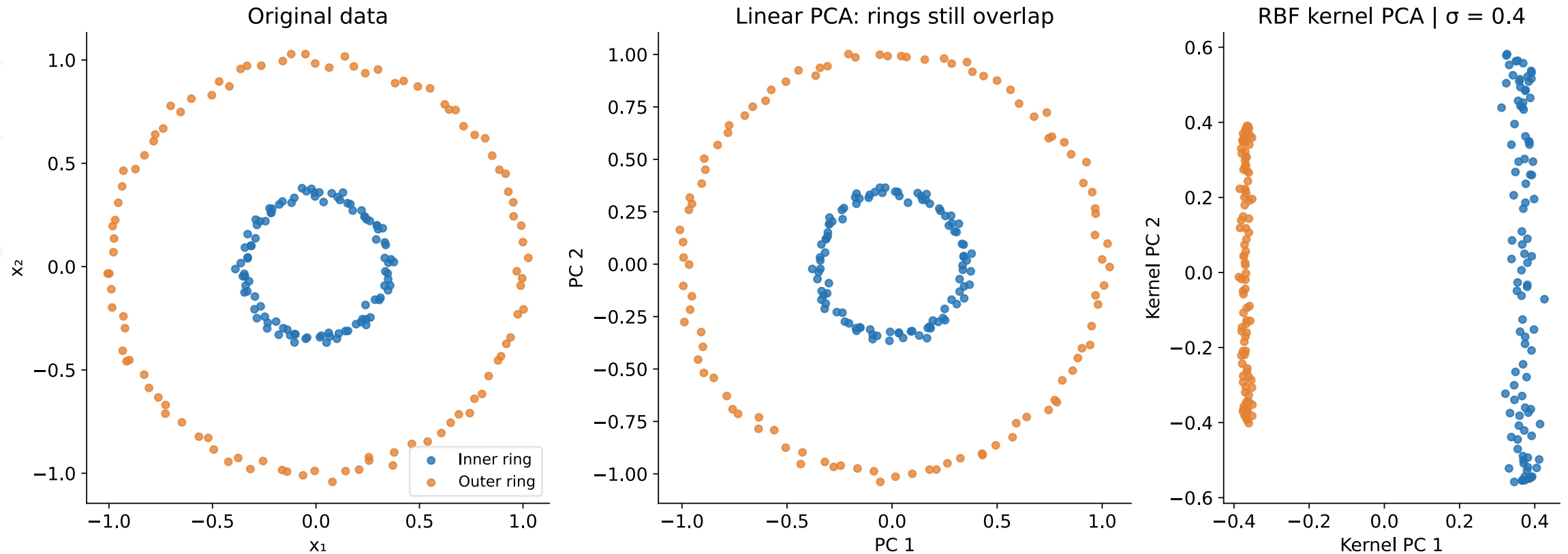
Summary of Kernel PCA

1. Pick a kernel function $k: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$
2. Construct the normalized matrix of the data $\{x^{(1)}, x^{(2)}, \dots, x^{(n)}\} \in \mathbb{R}^d$
$$\tilde{K} = K - \frac{1}{n} K \mathbf{1}_{n \times n} - \frac{1}{n} \mathbf{1}_{n \times n} K + \frac{1}{n^2} \mathbf{1}_{n \times n} K \mathbf{1}_{n \times n}$$
3. Solve an eigenvalue problem
$$\tilde{K} \alpha^j = n \lambda_j \alpha^j \quad \text{where } \alpha^{j \top} K \alpha^j = 1$$
4. For any data point x (new or old), we can represent it as

$$z = \begin{pmatrix} k_x^\top \alpha^1 \\ k_x^\top \alpha^2 \\ \dots \\ k_x^\top \alpha^n \end{pmatrix}$$

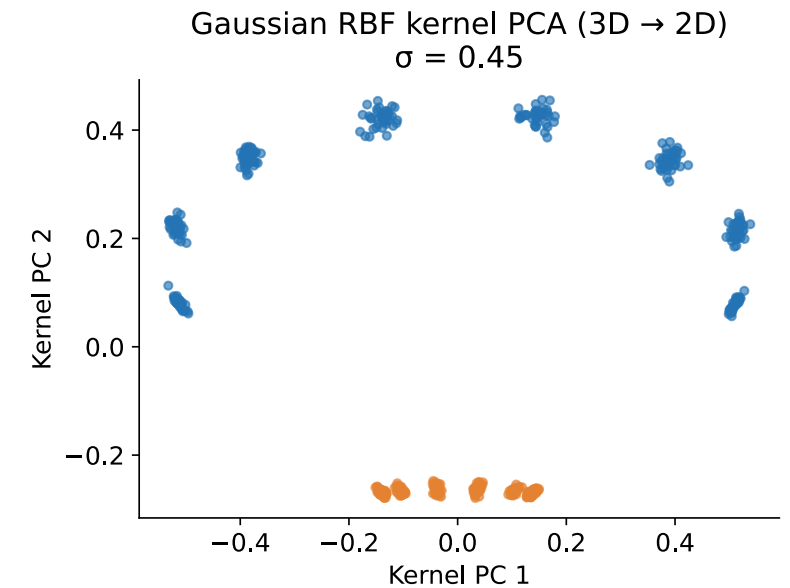
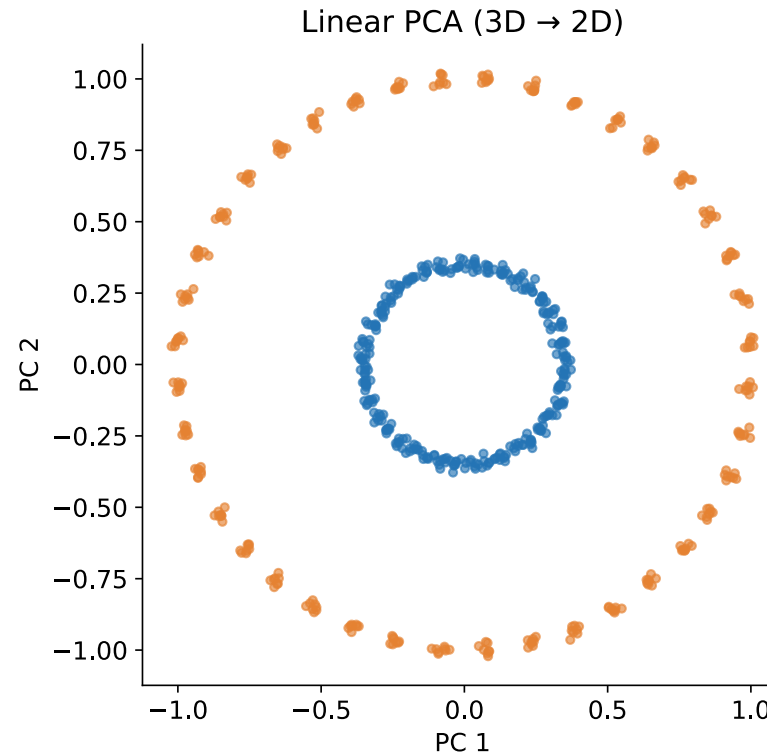
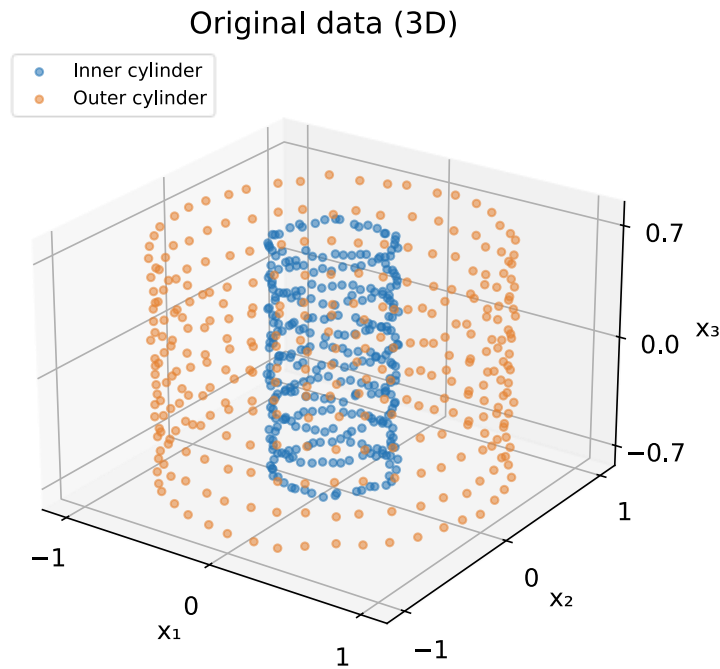
Kernel PCA Example

Colors are for interpretation only; neither PCA uses the ring labels



Kernel PCA Example

Nested cylinders: colors identify surfaces; PCA uses only the coordinates



Kernel PCA Example

- Reduce dimension and reconstruct back

MNIST denoising: 784 pixels $\rightarrow$ 64 kernel components $\rightarrow$ 784 pixels

